

CLEANCORE EXTERNAL EVALUATION — DEFENSE PLAN

Algorithmic & Software Engineering Strategy Guide

⚠ IF YOU HAVE ONLY 2–3 HOURS, GO STRAIGHT TO §19 AND §20. The 48-hour schedule in §12 is obsolete. The rest of this document is reference material — §19 is the new core.

Context

The external examiner has pre-loaded biases from the internal committee feedback (scheduling, truck capacity, notifications, data collection, bin lifecycle, real-time monitoring, background study). The `Clean_Core_Final_Addendum.pdf` gives you the strategic narrative; this plan is the **code-level companion** — every defense lands on a real file:line in your repo. Pair this with the existing `CleanCore_External_Evaluation_Study_Plan_AI.pdf` which already covers the ML side (Parts A–H). **This document covers what the AI plan does not: routing, capacity, notifications, scheduling, bin lifecycle, and how to deflect unfair attacks.**

Goal: by the end of 48 hours you can (1) recite where every feature lives, (2) walk through 2-opt by hand, (3) handle every committee comment with a 2-sentence defense + code pointer, (4) edit anything the examiner asks you to change live.

§0 — The One Paragraph You Memorize First

CleanCore is a ****worker-level routing and AI-monitoring layer**** that sits on top of city-scale fleet logistics (which the addendum explicitly scopes out to commercial APIs like HERE Maps/DHL). A FastAPI backend (`Backend/main.py`) loads two fine-tuned YOLOv8 models (fill + waste) on boot, exposes a `POST /analyze/` endpoint that IoT cameras push images to, classifies each bin's status against admin-configurable thresholds in `settings/main`, and on critical fill auto-attaches the bin to a matching active route or FCMs the responsible area worker. Route generation is a Greedy Nearest Neighbor seed refined by 2-opt local search over Haversine distance, run on demand from the worker's Flutter app. The admin panel (React + Firestore `onSnapshot`) gives live counts, anomaly feed, AI insights, and a settings page that retroactively reclassifies every bin when thresholds change. The dataset is 895 proprietary IoT-camera images from three European operators; transfer learning from `yolo8n.pt` reaches $mAP50 = 0.971$ on an unseen-camera hold-out.

§1 — Examiner Comments → Defense Mapping

Each row: examiner concern → 1-sentence defense → exact code pointer.

#	Examiner Comment	Defense (1 line)	Code Anchor
1	"Specify a proper time for taking pictures of bins"	"The hardware is battery-powered IoT, capture cadence is firmware-side (industry partner constraint, see addendum §2); the collection-time scheduling lives in the worker shift system."	Backend/main.py: 1774-1810 /admin/schedule-shift
2	"Check for truck capacity and cancel task"	"Capacity gating is fleet-level, scoped out per addendum §1. At the worker level we enforce per-stop concurrency locks (one truck per bin) and quiet-hours filtering for noisy waste streams."	Locks: Backend/main.py: 864-868 Quiet hours: 453-456 , 815-817
3	"Strong background study lacking"	"See §9 below — Bigbelly, Enevo, SmartBin precedents; we extend with cross-device hold-out validation that none of the cited papers attempt."	§9 of this plan
4	"Truck capacity handling and route assignment criteria not clearly defined"	"Route assignment uses three explicit criteria: (a) area-hierarchy match , (b) waste-type stream match with wildcards , (c) fill ≥ admin-set collection threshold , all gated by quiet-hours regulation."	Backend/main.py: 783-817
5	"Notification mechanisms for filled bins in different city areas"	"Two-tier FCM: critical bin in an active route's area → driver gets 'Urgent Pickup Added'; orphan critical → all area workers get 'Start a Route'. Batched on threshold changes to avoid spam."	Backend/main.py: 283-301 , 393-403 , 1749-1761
6	"Data collection should be automatic from all bins"	"It is. /analyze/ is the single ingestion point, hit by IoT cameras directly. The Flutter worker app has no manual image-upload path — that was an explicit removal."	Backend/main.py: 467-631 ; confirm: no image_picker in Flutter app views.
7	"Process for adding/removing bins, empty installs unclear"	"Admin panel Bins page → upload first photo → /analyze/ creates the doc; the initial scan is the bin's birth certificate . Delete cascades through every active route."	Add: Bins.tsx: 344-388 Delete: Backend/main.py: 1181-1206
8	"Timer/scheduling for periodic data collection"	"Capture interval is firmware-side on the IoT device (addendum §2 — battery preservation). On the backend, shift scheduling for collection windows is at /admin/schedule-shift , and quiet-hours regulation prevents Glass/Metal collection 23:00–05:00."	Backend/main.py: 1774-1810 ; Backend/main.py: 453-456
9	"Real-time monitoring; is truck already filled?"	"Admin dashboard has 10+ concurrent Firestore onSnapshot listeners showing live bin counts, active routes, completed stops. Driver progress streams via location updates."	Dashboard.tsx: 115-266 ; Backend/main.py: 890-970

§2 — Code Location Index (one-page cheat sheet)

Backend (Backend/main.py)

Concern	Function	Lines
FCM sender	<code>_send_fcm</code>	40-50
Threshold read	<code>_get_thresholds</code>	199-213
Status classification	<code>_classify_status</code>	216-222
Find area workers	<code>_find_workers_for_area</code>	259-280
Orphan critical FCM	<code>_notify_orphan_critical_bin</code>	283-301
Urgent attach to route	<code>_try_attach_urgent_bin</code>	304-404
Haversine distance	<code>_haversine_km</code>	408-414
Greedy Nearest Neighbor	<code>_greedy_nearest_neighbor</code>	416-425
2-opt refinement	<code>_two_opt_improve</code>	427-442
Total route km	<code>_total_route_km</code>	444-451
Quiet hours	<code>_is_quiet_hours</code>	453-456
AI analysis ingest	POST /analyze/	467-631
Re-scan existing bin	POST /rescan-bin/{bin_id}	637-711
Route generation	POST /optimize-route	741-870
Worker location ping	POST /update-worker-location	890-896
Complete a stop	POST /complete-stop	899-970
Delete bin (cascade)	DELETE /admin/delete-bin/{bin_id}	1181-1206
Save thresholds + recompute	POST /admin/save-thresholds	1700-1769
Schedule shift	POST /admin/schedule-shift	1774-1810

Admin Panel & Flutter App

App Block	Concern / File	Lines / Scope
Admin Panel	Live dashboard listeners / <code>Dashboard.tsx</code>	115-266
Admin Panel	Bins list + Add/Edit/Delete / <code>Bins.tsx</code>	214-466

App Block	Concern / File	Lines / Scope
Admin Panel	Active routes monitor / <code>Routes.tsx</code>	63-96
Admin Panel	Threshold UI + save / <code>Settings.tsx</code>	100-150
Admin Panel	Anomaly live feed / <code>AnomalyFeed.tsx</code>	45-77
Admin Panel	AI insights / <code>AIInsightsPanel.tsx</code>	19-108
Flutter App	FCM setup, token write, bg handler / <code>main.dart</code>	7-47
Flutter App	OSRM polyline fetch (display only) / <code>route_service.dart</code>	7-72
Flutter App	Live route & orphan listeners / <code>route_job_screen.dart</code>	54-95

§3 — Route Optimization Deep Dive

§3.1 — The full pipeline in one paragraph

When a worker taps "Start Route", the Flutter app POSTs to `/optimize-route` with the worker's depot GPS. The endpoint at `Backend/main.py:741-870` loads the worker's `assignedArea` + `assignedWasteType`, reads admin thresholds from `settings/main`, streams every bin with `fillLevel ≥ collectionThreshold`, filters by (i) not already locked, (ii) not hazardous-flagged, (iii) area hierarchy (word-boundary match), (iv) waste-type stream (wildcards Mixed/All/Any/General), and (v) quiet-hours regulation. The filtered stops go through Greedy Nearest Neighbor seeded at the depot, then 2-opt if there are >2 stops, followed by a return-to-depot total distance, fuel, and CO₂ computation. The route is persisted to Firestore `routes/{uuid}` and every bin is atomically batch-locked with `isLocked=true, routeId=<uuid>`.

§3.2 — Greedy Nearest Neighbor (the seed)

```
function greedy_nearest_neighbor(stops, depot):
    unvisited = copy(stops)
    ordered = []
    current = depot
    while unvisited:
        nearest = argmin over s in unvisited of haversine(current, s)
        ordered.append(nearest)
        current = nearest
        unvisited.remove(nearest)
    return ordered
```

- **Complexity:** $O(n^2)$ — n stops, each step scans all remaining unvisited elements.
- **Why it isn't optimal:** Greedy commits irrevocably; one early choice can lock you into a massive detour later.
- **Why use it anyway:** It is deterministic, fast, gets within ~25% of optimal on average, and provides a clean initial sequence for 2-opt.

§3.3 — 2-opt refinement

```
function two_opt_improve(stops, depot):
    best = stops
    best_dist = total_route_km(best, depot)
    passes = 0; improved = True
    while improved and passes < 3:
        improved = False; passes += 1
        for i in range(len(best) - 1):
            for j in range(i + 2, len(best)):
                candidate = best[:i] + reverse(best[i:j+1]) + best[j+1:]
                dist = total_route_km(candidate, depot)
                if dist < best_dist - 0.001: # 1-metre threshold
                    best, best_dist, improved = candidate, dist, True
    return best
```

What "2-opt" literally means: At each step, pick two edges in the tour and replace them with two different edges. Mathematically, it replaces `(A→B)` and `(C→D)` with `(A→C)` and `(B→D)`, reversing the segment between B and C. If a tour crosses itself, reversing the segment uncrosses it, shortening the total Haversine distance.

§3.3.1 — Worked example

Stops at coordinates: depot $D = (0,0)$, $A = (1,0)$, $B = (2,1)$, $C = (1,2)$, $E = (2,0)$. Suppose the greedy seed produces the tour: $D \rightarrow C \rightarrow A \rightarrow E \rightarrow B \rightarrow D$ ($\text{ext}\{Total\} = 8.48$). 2-opt swaps with $i=2$, $j=4$ (reversing segment $A,E,B \rightarrow B,E,A$) results in: $D \rightarrow C \rightarrow B \rightarrow E \rightarrow A \rightarrow D$. Distance updates: $D \rightarrow C = 2.24$, $C \rightarrow B = \sqrt{2} \approx 1.41$, $B \rightarrow E = 1.00$, $E \rightarrow A = 1.00$, $A \rightarrow D = 1.00$. $\text{ext}\{New\ Total\} = 6.65$. This provides a clear **improvement of 1.83 km** by uncrossing an X-shape configuration.

§3.3.2 — Complexity and Algorithm Zoo

- **Complexity:** The double loop does $O(n^2)$ segment-reversal candidates, costing $O(n)$ to evaluate distance, resulting in $O(n^3)$ per pass. Pass cap is locked at 3 for predictable worst-case bounds.

Algorithm	Idea	Cost	Quality vs Optimal
Brute Force	Try all $n!$ permutations	$O(n!)$	Optimal
Held-Karp DP	Dynamic programming	$O(2^n n^2)$	Optimal
Greedy NN	Always nearest unvisited	$O(n^2)$	$\sim 1.25\times$ optimal
2-opt	Local edge swap	$O(n^3)$ per pass	$\sim 1.05\times$ optimal typical
Lin–Kernighan	Adaptive k-opt	$O(n^{2.2})$ heuristic	State of the art

§3.4 — Distance metric: Haversine vs OSRM

The Backend uses **Haversine** for ordering because over a worker-level radius (~ 10 km) road-network topology rarely flips relative nearest-neighbor constraints. The Flutter app fetches **OSRM road polylines for display only**. Computing an $O(n^2)$ matrix via external OSRM API calls on every mobile request would introduce unacceptable latency.

§4 — Truck Capacity Defense (The Strategic Pivot)

CRITICAL TRUTH: The codebase does *not* structurally limit route sizes based on cumulative volumetric payload. Do not pretend this feature exists. Defend via architectural scoping.

Defense Script: "Macro-level fleet management (truck volumetric caps, multi-vehicle dispatch matrices) is intentionally scoped out and relegated to commercial APIs like HERE Maps or DHL fleet platforms. CleanCore is the **micro-level worker optimization layer** underneath. Given an assigned area depot, worker session, and specific waste stream, we generate the optimal sequential collection string."

If pressed on how to easily implement it, explain: "It requires three additions: (a) expose a `capacity_liters` field on the worker/vehicle profile, (b) accumulate expected bin volumes during the greedy iteration loop and break when the threshold is reached, (c) modify the urgent auto-attach route routine to drop down to the orphan FCM warning stream if adding the stop pushes the active truck over its volume limit."

§5 — Notifications & Real-Time Monitoring Defense

§5.1 — Two-Tier FCM Notification Engine

When a critical fill level event is processed via `/analyze/` or an admin threshold update, the system routes notifications through a deterministic logic tree:

- **Tier 1 (Active Route Intercept):** If the critical bin falls inside the `assignedArea` and waste stream of a driver currently running an active route, `_try_attach_urgent_bin` locks the bin, appends it dynamically to the route stops, and fires an FCM push to that driver: *"Urgent Pickup Added"*.
- **Tier 2 (Orphan Critical broadcast):** If no active matching route is found, `_notify_orphan_critical_bin` falls back to scanning for all off-duty or unassigned area workers using a word-boundary regex match on their `assignedArea` profile, sending a broad push: *"Urgent Bin — Start a Route"*.

§5.2 — Real-Time Dashboard Listeners

The React Admin Console establishes **10+ concurrent Firestore onSnapshot sockets** inside `Dashboard.tsx`. This eliminates polling mechanisms. Telemetry feeds for total active bins, active workers, pickup loops, real-time anomalies, and driver location metrics update with sub-second latency directly from backend writes.

§6 — Scheduling & Data Collection Defense

The Core Distinction: Image capture cadence is decoupled entirely from the server framework. It lives **firmware-side on the battery-powered IoT hardware** to optimize power draw and prevent aggressive battery exhaustion. The server remains a passive ingestion service exposing `POST /analyze/`.

The time-sensitive management that CleanCore *does* own is operational routing constraints. The system manages **collection-window worker shifts** via `/admin/schedule-shift` and strictly implements **regulatory quiet-hours validation** inside `Backend/main.py:453-456`, which completely strips Glass and Metal collection blocks between 23:00 and 05:00 to mirror regional municipal noise frameworks (e.g., German `TA Lärm` regulations).

§7 — Bin Lifecycle Management

- **Adding Bins:** Admin Panel `Bins.tsx` uploads the physical location coordinate matrix along with an initial setup photo. The system routes this to `/analyze/`. The resulting YOLO inference operates as the bin's formal birth certificate, setting its initial status to `normal` (the model has a 1.0 recall rating on completely empty configurations).
- **Removing Bins:** `DELETE /admin/delete-bin/{bin_id}` runs a safe cascading delete sequence. It sweeps through active and pending routes, cleans the stop out of existing arrays, decrements the `totalStops` count, and triggers an auto-complete if the target bin was the last remaining stop.

§15 — Dr. Usman Habib: Targeted Examiner Strategy

Dr. Usman Habib is the Professor and Head of Software Engineering at FAST-NUCES Islamabad. His PhD from TU Wien directly addressed automated fault detection in building data using sensor networks. He will review this architecture with strict academic and industrial rigor.

§15.2 — The Common-Ground Hook (Memorize the First 30 Seconds)

"Sir, before I dive in — I want to acknowledge that the problem class CleanCore tackles is structurally similar to your TU Wien work on automated fault detection in building sensor data. Both projects do supervised ML inference from low-frequency, hardware-constrained IoT sensor snapshots; both validate via cross-device hold-out rather than random splits; both ship through industrial partnerships rather than scraped public data. We treated fill level as a discrete classification problem rather than continuous regression, in the same spirit that building-fault-detection literature treats HVAC state as discrete operational modes. So if some of the methodological choices feel familiar, that's deliberate."

§15.4 — Expected Architecture & Engineering Defenses

- **The Single Monolithic File (`main.py`):** "The 1,707 lines are clean and split into modular functional blocks (Firebase init, optimization helpers, ingestion endpoints). For the scope of this FYP, keeping the execution layer contained inside a predictable file boundary simplifies runtime verification. A multi-package architectural split is the first item on our v2 refactoring roadmap."
- **Testing Gaps & Test Cases:** Point directly to the Testing Chapter in `Docs/Report Clean Core.pdf`. "The test cases are thoroughly mapped out as functional compliance parameters in the report. We verified code correctness against these explicit scripts. They operate as our functional specifications; converting them directly into automated pytest fixtures is our next technical milestone."

§16 — Targeted Mock Viva: Habib-Specific Matrix

Q: Why use YOLOv8 instead of an autoencoder-based anomaly detector for tracking bin fill states?

A: Autoencoders excel at binary anomaly flags (normal vs defective), but municipal logistics require graduated volume metrics (Empty, Half-Full, Full, Overflowing) to run adaptive routing thresholds. YOLOv8 gives us discrete class sorting alongside localization capabilities out of the box.

Q: Did you evaluate via leave-one-operator-out cross-validation?

A: We isolated our smallest and most visually divergent deployment stream (UKConnect) entirely as a strict cross-device validation hold-out. Running standard LOOO folds would heavily strip our 'Overflowing' labels, as those samples were clustered within a single operator's dataset, preventing the model from learning that specific class configuration.

Q: How does the system scale to 12,000 bins?

A: YOLOv8s executes sub-second on standard CPU architectures. Quantizing down to an ONNX layout lowers processing requirements to ~100ms. At a city footprint of 12,000 bins transmitting four times per day, average server demand is only ~0.55 inferences per second, which is easily sustained by a basic virtual machine instance. Firestore seamlessly handles document scales of this volume, and our 2-opt route partitions naturally by geographical area boundaries.

§19 — Emergency 2-Hour Cram Schedule

When pressed for time, drop long-form analysis and execute this sequence:

Hour 1: Core Controls & Hook Drill

- **00–05 min:** Practice reciting the §15.2 Common-Ground Hook aloud three times.
- **05–20 min:** Memorize §0 (The One Paragraph Elevator Pitch) and look over the §1 Comment Mapping table.
- **20–45 min:** Open your copy of the FYP Report. Find and memorize three functional test case IDs from the Testing section.
- **45–60 min:** Trace the route filter flags inside `Backend/main.py:741-870` to see how quiet hours and area assignments block invalid stops.

Hour 2: Demo Verification & Code Edits

- **00–20 min:** Spin up your environments (FastAPI, React dev server, Flutter workspace) and verify the 7-step demo story smoothly updates states live.
- **20–50 min:** Practice the code edits detailed in §20. Physically open your editor, change the 2-opt pass limit or the quiet-hours window parameters, confirm reload, and verify performance.
- **50–60 min:** Hard-code quick bookmarks into your editor for line 216 (classification logic), line 427 (2-opt function block), and line 1700 (threshold recomputations).

§20 — Live Code-Edit Playbook

If instructed to modify constraints live on screen, use these explicit blueprints:

Edit: Modifying 2-Opt Evaluation Passes

Navigate to `Backend/main.py:433`:

```
// Change loop constraint from 3 to 5
while improved and passes < 5:
```

Defense Note: "Increasing the pass cap allows 2-opt to search deeper for optimal edge crossings. For our average route scale of under 30 stops, convergence occurs quickly; expanding to 5 passes hardens route quality but slightly raises processing time during dense loops."

Edit: Tightening Quiet Hours Regulations

Navigate to `Backend/main.py:455`:

```
// Adjust from 23:00-05:00 to 22:00-06:00
return hour >= 22 or hour < 6
```

Defense Note: "This modification directly demonstrates our time-aware routing layer. Saving this instantly re-evaluates the active route pool, filtering out Glass and Metal routes an hour earlier to conform to strict quiet-hours policies."

Operational Recovery Script

If an update throws an unhandled exception or breaks active state parameters, remain completely calm and state: **"Let's safely roll back via a quick undo sequence. This showcases why production systems require automated integration test runners before moving code live."** Press `ctrl+Z`, save the script, and allow Uvicorn to restore the clean state configuration.